

시뮬레이션 데이터 구성

① 로그 더미데이터 먼저 준비

예:

- 09:00 wearable_log → 심박수 90
- 09:00 feeder_log → 급여 50g
- 09:00 water_log → 음수 120ml
- 09:00 camera_log → resting

② 그걸 바탕으로 09:00 state 생성

예:

- heart_rate = 90
- daily_feed_amount = 50
- daily_water_amount = 120
- last_behavior = RESTING

③ 10:00, 11:00도 같은 식으로 생성

이렇게 여러 시점의 state를 만들어둡니다.

④ 시뮬레이션 실행

- 09:00 state 처리 → 정상
- 10:00 state 처리 → 활동량 감소
- 11:00 state 처리 → 식사 누락 + 심박 이상 → alert 생성

즉,

하위 로그는 원재료,

state는 시점별 요약본,

시뮬레이션은 state 재생입니다.

시뮬레이션 필요 데이터

- `wearable_log` → 심박/체온/활동량 제공
 - `feeder_log` → 식사 정보 제공
 - `water_log` → 음수 정보 제공
 - `camera_log` → 행동/위치/배변 정보 제공
 - `state` → 위 4개를 합친 시점별 통합 상태
 - `event_log` → state를 보고 판단한 결과
 - `alert_log` / `action_log` → 후속 처리
-

1. 질문 정리

지금 질문은 이것입니다.

지금까지 정한 흐름을 바탕으로, 아래 8개 테이블의 **최종 컬럼 구성안**을 정리해달라는 것입니다.

- `wearable_log`
- `feeder_log`
- `water_log`
- `camera_log`
- `state`
- `event_log`
- `alert_log`
- `action_log`

그리고 **영문 컬럼명 옆에 반드시 한글 뜻을 괄호로 붙여서** 보여달라는 요청입니다.

2. 대답

A. 먼저 전체 방향

이번 프로젝트는 **부트캠프 수준에서 구현 가능한 범위**로 가야 하므로, 테이블은 **“시연이 가능할 만큼만”** 두는 것이 좋습니다. 흐름은 이렇게 유지하면 됩니다.

원천 로그 4개

- `state` 통합
 - `event_log` 판단 결과
 - `alert_log` 알림
 - `action_log` 조치
-

B. `wearable_log` (웨어러블 원천 로그)

심박, 체온, 활동량을 주는 테이블입니다.

- `id` (로그 번호)
- `pet_id` (반려동물 번호)
- `heart_rate` (심박수)
- `body_temp` (체온)
- `activity_count` (활동량 수치)
- `recorded_at` (기록 시각)
- `created_at` (생성 시각)

설명

여기는 웨어러블 장비가 보낸 원본 데이터입니다.

`state`의 건강 상태를 만드는 재료입니다.

C. `feeder_log` (사료급여기 원천 로그)

식사 정보를 주는 테이블입니다.

- `id` (로그 번호)
- `pet_id` (반려동물 번호)
- `feed_amount` (급여량)
- `feed_status` (급여 상태)
- `feed_mode` (급여 방식)

- `recorded_at` (기록 시각)
- `created_at` (생성 시각)

컬럼 뜻 보충

- `feed_status` (급여 상태)
→ 성공, 실패 같은 상태
- `feed_mode` (급여 방식)
→ 자동, 수동 같은 방식

설명

최근 식사 여부와 급여량을 판단하는 재료입니다.

D. `water_log` (음수기 원천 로그)

음수 정보를 주는 테이블입니다.

- `id` (로그 번호)
- `pet_id` (반려동물 번호)
- `water_amount` (음수량)
- `drink_detected` (음수 감지 여부)
- `recorded_at` (기록 시각)
- `created_at` (생성 시각)

설명

음수량과 최근 물 섭취 여부를 판단하는 재료입니다.

E. `camera_log` (카메라 원천 로그)

행동, 위치, 배변 관련 정보를 주는 테이블입니다.

- `id` (로그 번호)
- `pet_id` (반려동물 번호)
- `detected_behavior` (감지 행동)
- `location_zone` (위치 구역)
- `defecation_detected` (배변 감지 여부)
- `urination_detected` (배뇨 감지 여부)

- `captured_at` (촬영 시각)
- `created_at` (생성 시각)

설명

여기는 영상 자체를 저장하는 테이블이 아니라,
카메라가 감지했다고 가정한 결과값을 저장하는 테이블입니다.

즉,

- 무엇을 하고 있었는지
- 어디에 있었는지
- 배변/배뇨가 감지됐는지

를 `state`에 넘겨줍니다.

F. `state` (통합 상태 테이블)

원천 로그 4개를 합쳐 만든 시점별 상태입니다.

- `id` (상태 번호)
- `pet_id` (반려동물 번호)
- `heart_rate` (심박수)
- `body_temp` (체온)
- `activity_count` (활동량 수치)
- `daily_feed_amount` (일일 누적 급여량)
- `daily_water_amount` (일일 누적 음수량)
- `last_behavior` (최근 행동)
- `current_location` (현재 위치)
- `elimination_status` (배변 상태)
- `recorded_at` (상태 기준 시각)
- `processed` (처리 여부)
- `created_at` (생성 시각)

컬럼 뜻 보충

- `elimination_status` (배변 상태)

→ 정상, 없음, 이상징후 같은 값

- `processed` (처리 여부)

→ 이 state를 시뮬레이션에서 이미 사용했는지 표시

설명

이 테이블이 시뮬레이션의 핵심입니다.

시뮬레이션 시작 시 `processed = false` 인 상태를 시간순으로 하나씩 꺼내 처리하면 됩니다.

G. `event_log` (판단 결과 로그)

`state` 를 보고 판단한 결과를 저장합니다.

- `id` (이벤트 번호)
- `state_id` (상태 번호)
- `pet_id` (반려동물 번호)
- `event_category` (이벤트 구분)
- `event_code` (이벤트 상세 코드)
- `score` (점수)
- `severity` (위험 등급)
- `message` (설명 문구)
- `created_at` (생성 시각)

컬럼 뜻 보충

- `event_category` (이벤트 구분)
 - `HIGHLIGHT` (하이라이트용), `HEALTH` (건강용)
- `event_code` (이벤트 상세 코드)
 - 식사 누락, 심박 이상, 배변 이상 같은 구체 사건
- `severity` (위험 등급)
 - `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`

설명

이 테이블에서 점수와 등급이 계산됩니다.

즉, 판단 책임은 `state` 가 아니라 `event_log` 에 있습니다.

H. `alert_log` (알림 로그)

알림을 보낸 결과를 저장합니다.

- `id` (알림 번호)
- `event_id` (이벤트 번호)
- `pet_id` (반려동물 번호)
- `alert_type` (알림 종류)
- `alert_status` (알림 상태)
- `alert_message` (알림 문구)
- `sent_at` (발송 시각)
- `created_at` (생성 시각)

컬럼 뜻 보충

- `alert_type` (알림 종류)
→ 팝업, 배너, 경고창 등
- `alert_status` (알림 상태)
→ 발송 완료, 확인 대기, 확인 완료 등

설명

모든 이벤트가 여기로 오는 것은 아닙니다.

점수나 등급이 기준을 넘은 경우만 기록됩니다.

I. `action_log` (조치 로그)

알림 이후 취한 조치를 저장합니다.

- `id` (조치 번호)
- `alert_id` (알림 번호)
- `pet_id` (반려동물 번호)
- `action_type` (조치 종류)
- `action_mode` (조치 방식)
- `action_status` (조치 상태)
- `action_message` (조치 내용)

- `acted_at` (조치 시각)
- `created_at` (생성 시각)

컬럼 뜻 보충

- `action_type` (조치 종류)
→ 급여 실행, 물 공급, 상태 확인 요청 등
- `action_mode` (조치 방식)
→ 자동, 수동
- `action_status` (조치 상태)
→ 완료, 실패, 대기

설명

대시보드 버튼 클릭이나 자동 처리 결과가 여기 남습니다.

J. 최종 관계를 한 번에 정리하면

아주 짧게 보면 이렇게 됩니다.

- `wearable_log` → 심박/체온/활동량 제공
 - `feeder_log` → 식사 정보 제공
 - `water_log` → 음수 정보 제공
 - `camera_log` → 행동/위치/배변 정보 제공
 - `state` → 위 데이터를 합친 시점별 상태
 - `event_log` → 상태를 해석한 판단 결과
 - `alert_log` → 실제 알림 기록
 - `action_log` → 알림 뒤 조치 기록
-

K. 실무적으로 하나 더 정리

지금 단계에서는 너무 많은 컬럼을 넣는 것보다 컬럼 의미가 분명한 게 더 중요합니다.

특히 이번 설계에서 중요한 기준은 이겁니다.

- 원천값은 원천 로그에 둔다
- 통합 상태는 `state` 에 둔다
- 점수와 등급은 `event_log` 에 둔다

- 알림은 `alert_log`
- 조치는 `action_log`

이 선만 안 흐리면 발표 때도 설명이 깔끔합니다.

3. 결론 요약

최종적으로는

원천 로그 4개(`wearable_log` , `feeder_log` , `water_log` , `camera_log`)가 데이터를 제공하고, 이를 합쳐 `state` 를 만들고, `state` 를 바탕으로 `event_log` 에서 점수와 등급을 판단한 뒤, 필요하면 `alert_log` , `action_log` 로 이어지는 구조가 가장 현실적입니다.

또한 `camera_log` 는 단순 미디어용이 아니라, 행동·위치·배변 감지 정보를 `state` 에 공급하는 역할로 두는 것이 맞습니다.

신뢰도: 97/100

이 수치를 높이는 방법: 다음 단계에서 제가 이 컬럼안을 바탕으로 PK/FK(기본키/외래키)까지 붙인 최종 테이블 정의서 형식으로 한 번 더 다듬으면 됩니다.